

RestoConnect - Project Documentation

Summary:

RestoConnect is a restaurant management platform connecting customers, owners, staff, and administrators. It streamlines restaurant operations from discovery, ordering, and reservations to staff management and analytics.

1. System Overview

1.1 Purpose

- Customers can discover restaurants, browse menus, place orders, and make reservations.
- Restaurant owners can manage operations, staff, and track performance.
- Staff can handle orders, manage tables, and complete tasks.
- Admins oversee the platform, manage users, and approve restaurants.

1.2 Technology Stack

- **Backend:** Node.js, Express.js
 - **Database:** MongoDB with native driver
 - **Template Engine:** EJS
 - **Authentication:** Session-based with bcrypt
 - **Frontend:** HTML, CSS, JavaScript, Chart.js
-

2. User Roles & Access

- **Administrator:** Full platform access; manages users and restaurants.
- **Restaurant Owner:** Manages menu, staff, tables, and analytics.
- **Staff:** Handles orders, tables, and tasks for assigned restaurants.
- **Customer:** Browses restaurants, orders, makes reservations, and submits feedback.

Authentication:

- Registration validates inputs and hashes passwords.
 - Login verifies credentials and initiates sessions.
 - Role-based middleware ensures access control.
-

3. Database Architecture

Core Collections

- **Users:** Authentication credentials, roles, restaurant association.
- **Customers:** Profiles, contact info, order history, analytics.
- **Restaurants:** Details, menu, orders, reservations, tables, inventory, tasks, revenue.
- **Dishes:** Menu items with name, price, description, and image.
- **Orders:** Customer orders with dishes, status, total, and timestamp.
- **Feedback:** Customer reviews linked to restaurant and customer.
- **Restaurant Requests:** Pending restaurant onboarding submissions.

Relationships

- Restaurants reference dishes and orders by IDs.
 - Orders reference dish IDs and customer IDs.
 - Staff and owner users reference their restaurant ID.
-

4. Key Features

Customer

- Restaurant discovery with search and location filtering.
- Menu browsing and cart management.
- Place orders, make reservations, or both.
- View order history and submit feedback.

Restaurant Owner

- Dashboard with analytics: total orders, revenue, top dishes.
- Menu and dish management.
- Staff creation and management.
- Table allocation and reservation tracking.
- Task assignment and tracking.

Staff

- Dashboard for pending orders, reservations, and tasks.
- Update order status.
- Allocate tables for reservations.
- Complete tasks assigned by owner.

Administrator

- Manage users and roles.
 - Approve or reject restaurant requests.
 - Add, edit, or delete restaurants.
 - View system-wide statistics and revenue.
-

5. Dynamic and Async Features

- **Dynamic DOM Updates:**
 - Admin dashboard updates stats without page reload.
 - Home page filters restaurants based on input/location.
 - Login/signup toggles forms dynamically.
- **Async / API Calls:**
 - Admin CRUD operations via AJAX.
 - Owner manages staff and tables via AJAX.
 - Customer fetches restaurants asynchronously after login.

6. Security

- Passwords hashed with bcrypt.
- Session cookies expire after inactivity; role-based access control.
- Input validation for required fields, emails, numeric values, and dates.

9. Workflows

- **Order Workflow:** Customer adds dishes → confirms order → system updates orders, restaurant data, reservations, and cart.
- **Reservation Workflow:** Customer books table → staff allocates table → reservation updated in restaurant collection.
- **Staff Task Workflow:** Owner creates task → staff completes/deletes task → updates reflected immediately.
- **Restaurant Onboarding:** Owner requests approval → admin reviews → approves/rejects → creates restaurant and owner account if approved.

10. References

- **Key Files & Functions:** See README section for `/source`, `/public/js`, `/views` scripts.
 - **Database Dump:** `mongo_dump/` folder.
 - **Test Cases:** `test_plan.md`
 - **Task Assignment:** `task_assignment.md` or CSV
-